Nome: Luis Augusto de Souza
Aula 10 - Exrcícios Recursividade

Entrega: até 22/10 às 17h

1. As questões devem ser resolvidas na **Linguagem C**.
2. É terminantemente proibido o uso de laços de repetição (`for` , `while` ou `do-while`) dentro da lógica principal de cálculo; a solução deve ser puramente recursiva.
3. Identifique claramente em comentários no código a linha correspondente ao **Caso Base** e a linha do **Passo Recursivo**.

EXERCÍCIO 1 Tamanho de uma String Recursivo (tamstring)

Descrição: Escrever uma função recursiva que retorna o tamanho de uma string em C, utilizando a assinatura `int tamstring(char s[])` . Lembre-se de que o caractere terminador nulo `'\0'` marca o fim da string e serve como caso base natural.

Exemplos: `tamstring("algoritmos")` → 10 | `tamstring("ifes")` → 4

EXERCÍCIO 2 Contagem de Ocorrências de Caractere (caract)

Descrição: Fazer uma função recursiva que conta o número de ocorrências de um determinado caractere em uma string, utilizando a assinatura `int caract(char c, char s[])` .

Exemplos: `caract('a', "arara")` → 3 | `caract('x', "computador")` → 0

EXERCÍCIO 3 Inversão de uma String (inverte_string)

Descrição: Desenvolva uma função recursiva `void inverte_string(char s[], int inicio, int fim)` que receba uma string e os índices inicial e final, invertendo os caracteres da string "in place" de forma puramente recursiva.

Exemplos: `"ifes"` → `"sefi"` | `"recursao"` → `"oasrucer"`

EXERCÍCIO 4 Maior Elemento de um Vetor (maior_elemento)

Descrição: Construa a função recursiva `int maior_elemento(int v[], int n)` que receba um vetor de inteiros `v` e seu tamanho atual `n ≥ 1` , retornando o maior valor armazenado no vetor.

Exemplos: `v = {4, 12, 7, 9, 3}, n = 5` → 12

EXERCÍCIO 5 Verificação de Palíndromo (eh_palindromo)

Descrição: Escreva uma função recursiva `int eh_palindromo(char s[], int inicio, int fim)` que retorne `1` se a string for um palíndromo (lê-se igual de trás para frente, ignorando espaços) e `0` caso contrário.

Exemplos: `"arara"` → 1 | `"programacao"` → 0

EXERCÍCIO 6 Máximo Divisor Comum - MDC de Euclides (mdc)

Descrição: Implemente a função recursiva `int mdc(int a, int b)` utilizando o célebre Algoritmo de Euclides para calcular o Máximo Divisor Comum entre dois números inteiros não negativos `a` e `b` , sabendo que `mdc(a, 0) = a` e `mdc(a, b) = mdc(b, a % b)` .

Exemplos: `mdc(48, 18)` → 6 | `mdc(100, 25)` → 25

recursividade.h

```
#ifndef RECURSIVIDADE_H
#define RECURSIVIDADE_H

/* EXERCÍCIO 1: Tamanho de uma String Recursivo */
int tamstring(char s[]);

/* EXERCÍCIO 2: Contagem de Ocorrências de Caractere */
int caract(char c, char s[]);

/* EXERCÍCIO 3: Inversão de uma String (in place) */
void inverte_string(char s[], int inicio, int fim);

/* EXERCÍCIO 4: Maior Elemento de um Vetor */
int maior_elemento(int v[], int n);

/* EXERCÍCIO 5: Verificação de Palíndromo */
int eh_palindromo(char s[], int inicio, int fim);

/* EXERCÍCIO 6: Máximo Divisor Comum - MDC de Euclides */
int mdc(int a, int b);

#endif
```

utils.h

```
#ifndef UTILS_H
#ifndef UTILS_H
#define UTILS_H

int ler_inteiro(const char *mensagem);
void ler_string(const char *mensagem, char *dest, int tamanho);

#endif
#endif
```

recursividade.c

```c
#include "recursividade.h"
#include <stdio.h>

/* ========================================
 * EXERCÍCIO 1: Tamanho de uma String (tamstring)
 * ======================================== */
int tamstring(char s[]) {
    /* 1. Caso Base: o carater terminador nulo '\0' marca o fim da string */
    if (s[0] == '\0') {
        return 0;
    }
    /* 2. Passo Recursivo: 1 mais o tamanho do restante da string */
    return 1 + tamstring(s + 1);
}

/* ========================================
 * EXERCÍCIO 2: Contagem de Ocorrências (caract)
 * ======================================== */
int caract(char c, char s[]) {
    /* 1. Caso Base: fim da string alcançado */
    if (s[0] == '\0') {
        return 0;
    }
    int match = (s[0] == c) ? 1 : 0;

    /* 2. Passo Recursivo: soma o match atual com a contagem no restante */
    return match + caract(c, s + 1);
}

/* ========================================
 * EXERCÍCIO 3: Inversão de String (inverte_string)
 * ======================================== */
void inverte_string(char s[], int inicio, int fim) {
    /* 1. Caso Base: os índices cruzam-se ou encontram-se no centro */
    if (inicio >= fim) {
        return;
    }

    char temp = s[inicio];
    s[inicio] = s[fim];
    s[fim] = temp;

    /* 2. Passo Recursivo: aproxima os extremos em direção ao centro */
    inverte_string(s, inicio + 1, fim - 1);
}
```

```c
/* ========================================
 * EXERCÍCIO 4: Maior Elemento de um Vetor (maior_elemento)
 * ======================================== */
int maior_elemento(int v[], int n) {
    /* 1. Caso Base: vetor de tamanho 1 (n >= 1) */
    if (n == 1) {
        return v[0];
    }

    /* 2. Passo Recursivo: obtém o maior elemento no restante do vetor */
    int maior_restante = maior_elemento(v + 1, n - 1);

    return (v[0] > maior_restante) ? v[0] : maior_restante;
}

/* ========================================
 * EXERCÍCIO 5: Verificação de Palíndromo (eh_palindromo)
 * ======================================== */
int eh_palindromo(char s[], int inicio, int fim) {
    /* 1. Caso Base 1: os ponteiros cruzam-se ou coincidem */
    if (inicio >= fim) {
        return 1;
    }
    if (s[inicio] == ' ') {
        return eh_palindromo(s, inicio + 1, fim);
    }
    if (s[fim] == ' ') {
        return eh_palindromo(s, inicio, fim - 1);
    }
    /* Caso Base 2: carateres diferentes nas pontas */
    if (s[inicio] != s[fim]) {
        return 0;
    }

    /* 2. Passo Recursivo: avança ambos os limites para o interior */
    return eh_palindromo(s, inicio + 1, fim - 1);
}

/* ========================================
 * EXERCÍCIO 6: Máximo Divisor Comum - MDC (mdc)
 * ======================================== */
int mdc(int a, int b) {
    /* 1. Caso Base: mdc(a, 0) = a */
    if (b == 0) {
        return a;
    }
```

```
    /* 2. Passo Recursivo: mdc(b, a % b) */
    return mdc(b, a % b);
}
```

utils.c

```
#include "utils.h"
#include <stdio.h>
#include <string.h>

int ler_inteiro(const char *mensagem) {
    int valor;
    printf("%s", mensagem);
    scanf("%d", &valor);

    int c;
    while ((c = getchar()) != '\n' && c != EOF);

    return valor;
}

void ler_string(const char *mensagem, char *dest, int tamanho) {
    printf("%s", mensagem);
    fgets(dest, tamanho, stdin);

    size_t len = strlen(dest);
    if (len > 0 && dest[len - 1] == '\n') {
        dest[len - 1] = '\0';
    }
}
```

main.c

```
#include <stdio.h>
#include "recursividade.h"
#include "utils.h"

int main() {
    printf("=== TAREFA 17: RECURSIVIDADE - IFES COLATINA ===\n\n");

    /* Exercício 1: Tamanho de String */
    char str1[100];
    ler_string("--- Exercicio 1: Tamanho de String ---\nDigite uma palavra: ", str1, sizeof(str1));
    printf("Resultado: tamstring(\"%s\") = %d\n\n", str1, tamstring(str1));

    /* Exercício 2: Contagem de Ocorrências de Caractere */
    char str2[100], caractere;
    printf("--- Exercicio 2: Ocorrencia de Caractere ---\n");
    ler_string("Digite uma palavra/frase: ", str2, sizeof(str2));
    printf("Digite o carater a ser contado: ");
    scanf(" %c", &caractere);
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
    printf("Resultado: caract('%c', \"%s\") = %d\n\n", caractere, str2, caract(caractere, str2));

    /* Exercício 3: Inversão de String */
    char str3[100];
    printf("--- Exercicio 3: Inversao de String (In Place) ---\n");
    ler_string("Digite uma palavra para inverter: ", str3, sizeof(str3));
    int len3 = tamstring(str3);
    if (len3 > 0) {
        inverte_string(str3, 0, len3 - 1);
    }
    printf("Resultado invertido: %s\n\n", str3);

    /* Exercício 4: Maior Elemento de um Vetor */
    printf("--- Exercicio 4: Maior Elemento de um Vetor ---\n");
    int vetor[] = {4, 12, 7, 9, 3};
    int n_vetor = 5;
    printf("Vetor padrão: {4, 12, 7, 9, 3} (Tamanho: %d)\n", n_vetor);
    printf("Resultado: maior_elemento = %d\n\n", maior_elemento(vetor, n_vetor));

    /* Exercício 5: Verificação de Palíndromo */
    char str5[100];
    printf("--- Exercicio 5: Verificacao de Palindromo ---\n");
    ler_string("Digite uma palavra/frase para testar: ", str5, sizeof(str5));
    int len5 = tamstring(str5);
    int palindromo = 0;
```

```
    if (len5 > 0) {
        palindromo = eh_palindromo(str5, 0, len5 - 1);
    }
    printf("Resultado: eh_palindromo = %d (%s)\n\n", palindromo, palindromo ? "Eh
palindromo" : "Nao eh palindromo");

    /* Exercício 6: MDC de Euclides */
    printf("--- Exercicio 6: MDC (Algoritmo de Euclides) ---\n");
    int num_a = ler_inteiro("Digite o primeiro numero (a): ");
    int num_b = ler_inteiro("Digite o segundo numero (b): ");
    printf("Resultado: mdc(%d, %d) = %d\n\n", num_a, num_b, mdc(num_a, num_b));

    return 0;
}
```